

🧠 Guide RAG / BrainVault pour débutant — comprendre et surveiller la partie IA de Riemann_Lab

Fichier : Guide-RAG-BrainVault-Debutant.md · **Dossier :** wiki (racine) **Branche :** master (wiki) · **Auteur :** hprzeta · **MAJ :** 2026-07-05

🔗 **Rôle de cette page.** Expliquer **sans jargon** comment fonctionne le cerveau IA du projet (le RAG « BrainVault » sur le SSD /mnt/vault_rag), ce que sont tous ces fichiers mystérieux (chroma.sqlite3, data_level0.bin, docstore.json...), et comment **surveiller** le système quand il tourne avec le script rag_monitor.py. Tous les exemples sont **réels** : ce sont les tests effectués les 4-5 juillet 2026 sur PC1.

1. C'est quoi un RAG ? (l'image du bibliothécaire)

RAG = *Retrieval-Augmented Generation* — « génération augmentée par récupération ».

Imagine un bibliothécaire (le modèle IA, ex. **mathstral**) très intelligent mais qui n'a **jamais lu tes documents**. Si tu lui demandes « quel est le seuil SEUIL_1NEWTON ? », il ne peut pas le savoir : cette valeur n'existe que dans TES fichiers (wiki, analyses, code).

Le RAG résout ça en 2 temps :

1. **Retrieval (récupération)** : avant de répondre, le système **cherche dans ta bibliothèque** les 3-5 passages les plus pertinents pour la question.
2. **Generation (génération)** : il donne ces passages au modèle IA avec la consigne « réponds à la question EN T'APPUYANT sur ces extraits ».

Résultat : le modèle répond avec **tes** données, à jour, et peut **citer ses sources**. Sans RAG, il inventerait (« hallucination »). Avec RAG, il consulte.

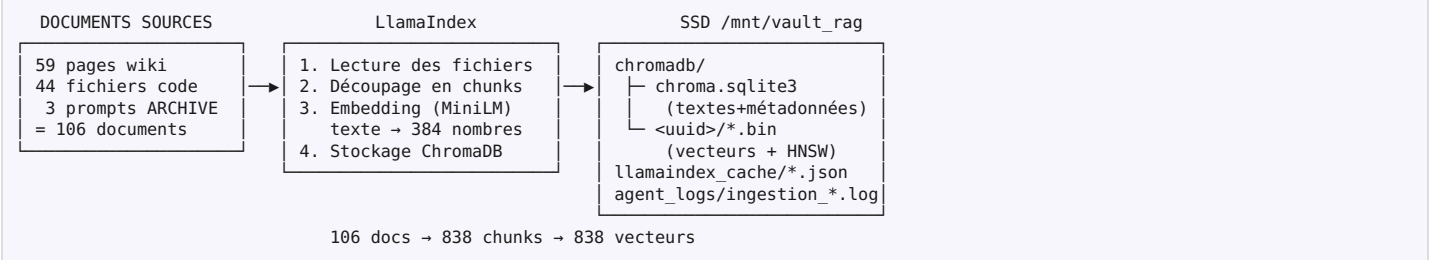
🔗 C'est exactement pour ça qu'on a corrigé les IP .94→.52 et posé les bandeaux ARCHIVE avant l'ingestion : **le RAG croit ce qu'on lui donne**. Documents faux = réponses fausses.

2. Le vocabulaire IA — traduit en français simple

Terme	Traduction simple	Dans Riemann_Lab
LLM	Le « cerveau » : un modèle de langage qui lit et écrit du texte	mathstral (spécialisé maths), deepseek-coder, qwen3 via Ollama
Ollama	Le logiciel qui fait tourner les LLM en local sur ton PC (pas de cloud)	modèles stockés sur /mnt/data/models_ia/ollama/
Chunk	Un morceau de document (~quelques paragraphes). On découpe car le LLM ne peut pas tout lire d'un coup	106 documents → 838 chunks
Embedding	La traduction d'un texte en nombres : une liste de 384 nombres qui capture le <i>sens</i> du texte	modèle all-MiniLM-L6-v2
Vecteur	Cette liste de nombres, vue comme une flèche dans un espace à 384 dimensions . Deux textes de sens proche → flèches qui pointent presque dans la même direction	1 chunk = 1 vecteur de 384 nombres
Similarité	La mesure « ces deux flèches pointent-elles dans la même direction ? » (angle entre vecteurs). C'est ça, la recherche sémantique : par le SENS, pas par les mots exacts	« seuil Newton » retrouve « SEUIL_1NEWTON=20000 » même sans mot commun
Base vectorielle	La base de données qui stocke tous les vecteurs et sait trouver les plus proches très vite	ChromaDB dans /mnt/vault_rag/chromadb/
Collection	Un « rayon » de la base vectorielle (un ensemble de chunks)	riemann_lab_corpus (838 chunks)
HNSW	L'astuce qui rend la recherche instantanée : un réseau de « raccourcis » entre vecteurs voisins, comme un GPS qui évite de tester toutes les routes	fichier link_lists.bin
LlamaIndex	Le chef d'orchestre : lit les documents, découpe en chunks, appelle l'embedding, remplit ChromaDB, puis pilote les requêtes	installé dans zeta_env (core 0.14.23)
Ingestion	L'opération « remplir la bibliothèque » : documents → chunks → vecteurs → ChromaDB	log : agent_logs/ingestion_20260705_140201.log
Corpus	L'ensemble des documents sources ingérés	wiki + code src/ + prompts ARCHIVE
Requête RAG	Question → embedding de la question → recherche des chunks proches → réponse du LLM avec ces chunks	test SEUIL_1NEWTON ✔

3. Le schéma complet — de tes documents à la réponse

Phase A — Ingestion (faite le 05/07/2026, ~5 min)



Test 2 — Ingestion complète (05/07, étape 3)

Indicateur	Valeur
Documents sources	106 (59 wiki + 44 code + 3 prompts ARCHIVE)
Chunks créés	838
Collection	riemann_lab_corpus
Exclusions respectées	Handoff.md , *.bak , logs
Durée	~5 min 32 s
Script réutilisable	scripts/rag_ingest_corpus.py (commit 61ca640)

Test 3 — Première requête RAG (05/07, étape 4)

Question : « Quel est le seuil SEUIL_1NEWTON et pourquoi cette valeur ? » → Le retrieval retrouve le **bon extrait** dans analyse_problemes_v13_v15.md ✓

C'est la preuve que la recherche **sémantique** marche : le système comprend que la question parle du seuil adaptatif de la Phase 2 Illinois, sans correspondance mot-à-mot exigée.

🛡️ **Test de contrôle qualité recommandé** : demander « Quelles sont les IP des 4 machines du cluster ? ». Réponse attendue : .24 / .52 / .22 / .54. Si .94 sort → une archive non taguée a fui dans le corpus (voir règle d'ingestion dans [[etat_rag_brainvault_20260704]]).

6. Surveiller le RAG — quoi observer et pourquoi

Quand le RAG « tourne » (ingestion en cours, ou requêtes), voici les **points de mesure** qui disent si tout va bien :

Point observé	Pourquoi c'est important	Signal d'alerte
Montage du SSD (mountpoint)	Leçon du 05/07 : SSD non remonté après reboot → écriture silencieuse sur le disque système	✗ NON MONTÉ → tout arrêter
Espace libre SSD et racine	Une ingestion qui écrit au mauvais endroit remplit /	racine qui se remplit pendant une ingestion
Taille de chromadb/	Doit croître pendant l'ingestion, être stable sinon	taille figée en pleine ingestion = blocage
Nombre de chunks par collection	La jauge de remplissage de la bibliothèque	compte qui n'augmente plus
Collections présentes	Détecter les résidus (hello-world) ou doublons	collections inattendues
Log d'ingestion (dernières lignes)	Erreurs de lecture, fichiers ignorés	ERROR , Traceback
Latence de requête	Santé de l'index HNSW	> 1 s pour le retrieval seul = anormal
RAM/CPU	8 Go de RAM : mathstral (7B) + embeddings peuvent saturer	swap intensif (voir \$RAM upgrade dans [[STACK]])

⚠️ **Piège découvert le 05/07 (à retenir absolument)** : le modèle d'embedding utilisé pour la **REQUÊTE** doit être **IDENTIQUE** à celui de l'**INGESTION** (all-MiniLM-L6-v2 ici). Deux modèles différents = deux « langues de nombres » incompatibles : les distances explosent (~1.6 au lieu de <0.8) et les extraits retrouvés n'ont **aucun rapport** avec la question. Symptôme typique : le retrieval « marche » techniquement mais renvoie n'importe quoi. C'est exactement ce que mesure --test-query du script ci-dessous.

Le script `scripts/rag_monitor.py` regroupe tout ça :

```
# photo instantanée de l'état du vault
python scripts/rag_monitor.py

# surveillance continue pendant une ingestion (rafraîchi toutes les 10 s)
python scripts/rag_monitor.py --watch 10

# + test de latence d'une vraie requête vectorielle
python scripts/rag_monitor.py --test-query
```

Exemple de sortie :

```
== RAG MONITOR - /mnt/vault_rag - 2026-07-05 14:30 ==
✓ SSD monté (/dev/sdb1) · libre : 227.8 G / 230 G
— Tailles —
chromadb           : 12.4 M
llamaindex_cache   : 3.1 M
agent_logs         : 48.0 K
— Collections ChromaDB —
riemann_lab_corpus : 838 chunks
— Dernier log —
ingestion_20260705_140201.log (5 dernières lignes)
...
Nodes (chunks) indexés : 838
```

7. Et après ? (où on en est dans l'Objectif 2)

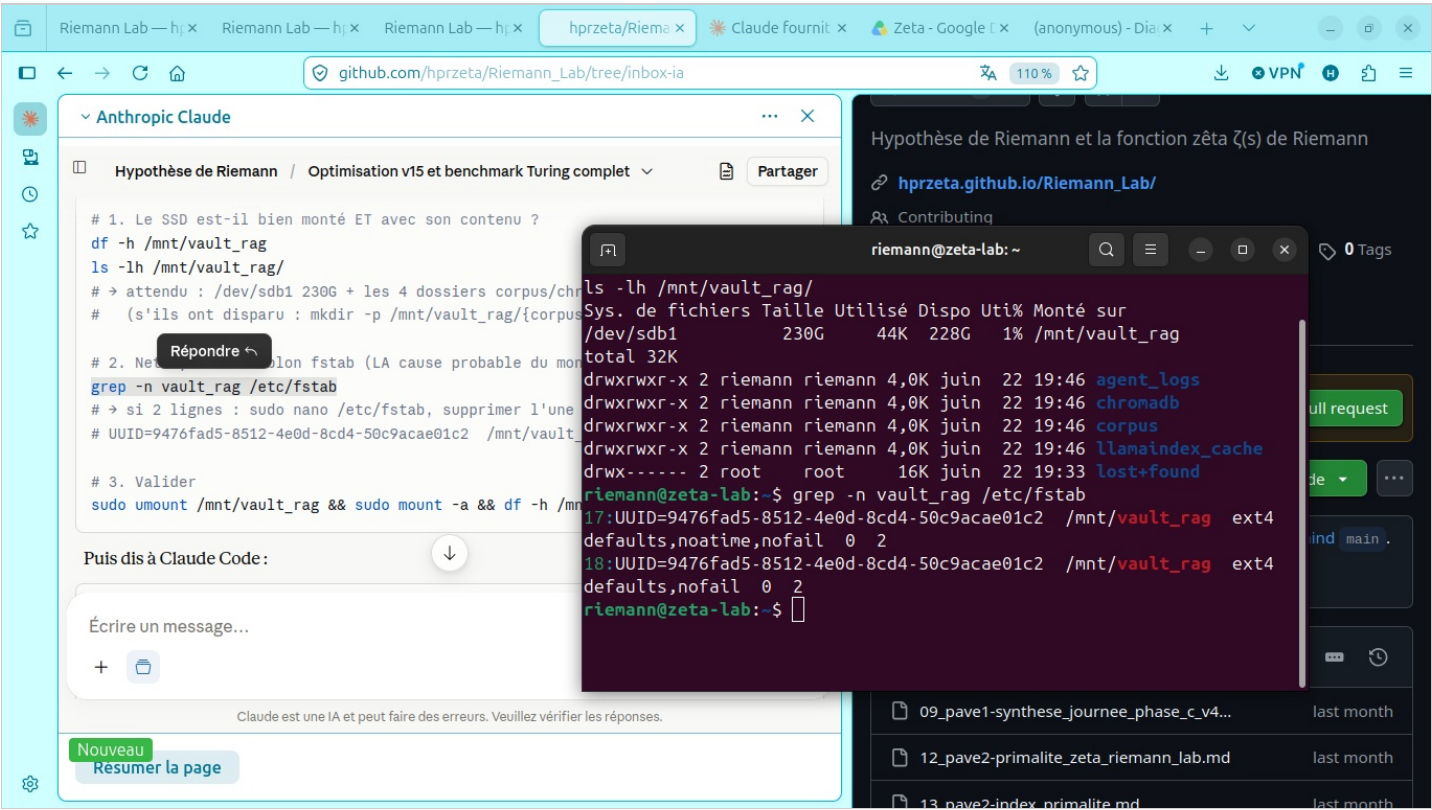
Étape du plan	État
1. Corpus poussé sur inbox-ia + manifeste	✔ 05/07
2. Stack installée + hello-world SSD	✔ 05/07
3. Ingestion complète (106 docs → 838 chunks)	✔ 05/07
4. Première requête RAG (SEUIL_1NEWTON)	✔ 05/07
5. Boucle complète avec Ollama/mathstral + parcours Skilljar (API → MCP → Subagents)	🚧 en cours

La suite logique : brancher mathstral sur ChromaDB pour des réponses génératives locales, puis construire l'agent autonome (MCP + subagents) qui interrogera ce vault pour publier rapports et résultats.

Voir aussi

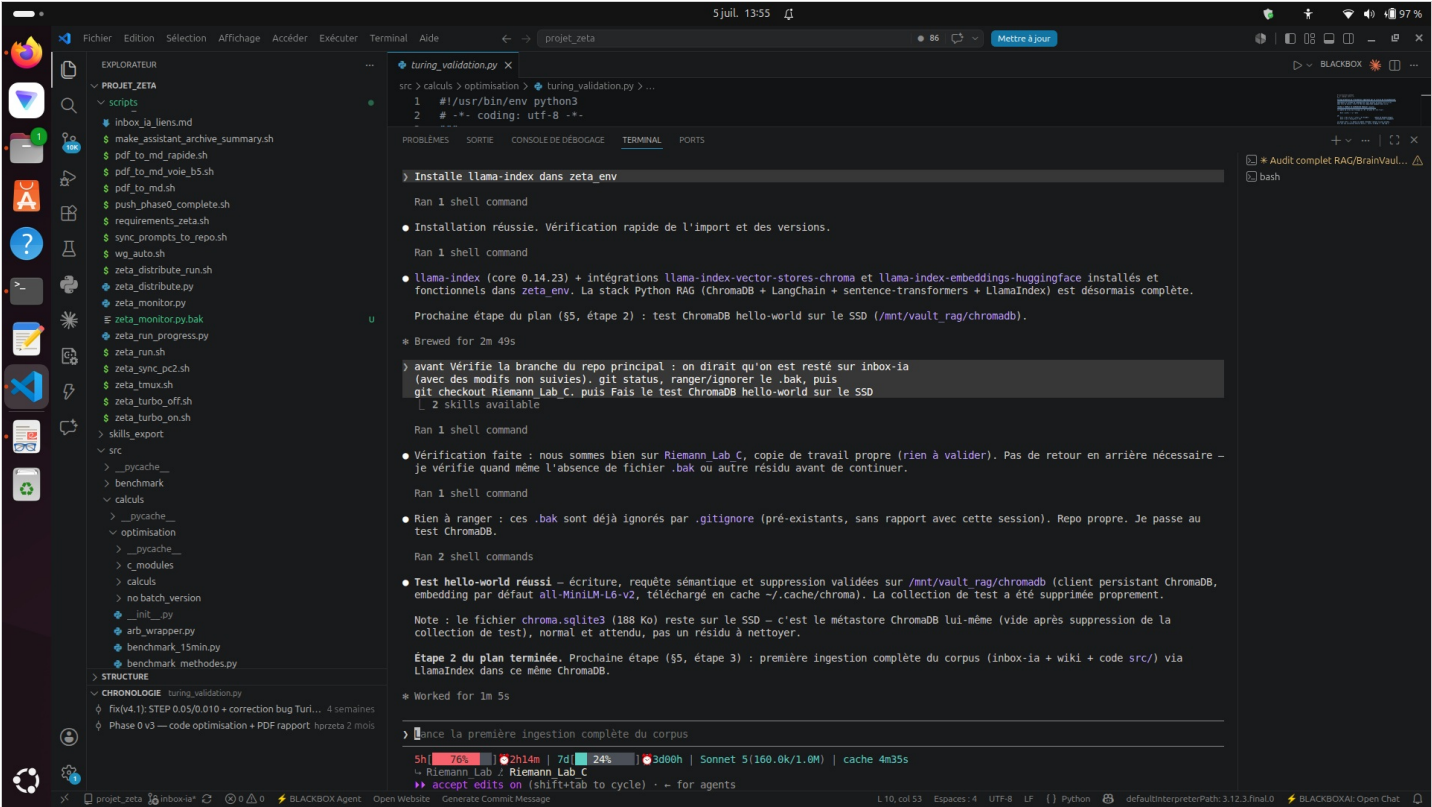
- [[etat_rag_brainvault_20260704]] — le bilan d'audit et le plan en 5 étapes
- [[ORGANISATION_FICHIERS]] — où vit quoi (règle d'ingestion, inbox-ia)
- [[STACK]] — outils, matériel, roadmap Objectif 2
- [[Bonnes-Pratiques-Claude-Code]] — gestion contexte/tokens

A.1 — Le SSD vault_rag monté, structure intacte, doublon fstab repéré



Le terminal montre `/dev/sdb1` (230 Go) monté sur `/mnt/vault_rag` avec les 4 dossiers du 22/06, et le `grep fstab` qui révèle la ligne en double (17 et 18) — corrigée ensuite.

A.2 — Test « hello-world » ChromaDB réussi + stack complète



llama-index installé, puis test hello-world : écriture, requête sémantique et suppression validées sur `/mnt/vault_rag/chromadb` — étape 2 du plan terminée.

A.3 — Ingestion complète : 106 documents → 838 chunks


```
5 jul. 14:16
Fichier Edition Sélection Affichage Accéder Exécuter Terminal Aide
EXPLORATEUR
PROJET_ZETA
scripts
inbox_la_liens.md
make_assistant_archive_summary.sh
pdf_to_md_rapide.sh
pdf_to_md_voie_b5.sh
pdf_to_md.sh
push_phase0_complete.sh
rag_ingest_corpus.py
requirements_zeta.sh
sync_prompts_to_repo.sh
wg_auto.sh
zeta_distribute_run.sh
zeta_distribute.py
zeta_monitor.py
zeta_monitor.py.bak
zeta_progress.py
zeta_run.sh
zeta_sync_p2c.sh
zeta_tmux.sh
zeta_turbo_off.sh
zeta_turbo_on.sh
skills_export
src
__pycache__
benchmark
calculs
__pycache__
optimisation
__pycache__
c_modules
calculs
no_batch_version
__init__.py
arb_wrapper.py
benchmark_15min.py
STRUCTURE
CHRONOLOGIE
turing_validation.py
fix(v4.1): STEP 0.05/0.010 + correction bug Turi... 4 semaines
Phase 0 v3 — code optimisation + PDF rapport hprzeta 2 mois
turing_validation.py X
src > calculs > optimisation > turing_validation.py > ...
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
PROBLEMES SORTIE CONSOLE DE DEBOGAGE TERMINAL PORTS
168 - fh.write(f"Nombre de nodes (chunks) : {len(index.docstore.docs)}\n")
112 + fh.write(f"Nombre de nodes (chunks) : {n_chunks}\n")
113 fh.write(f"Collection chromadb : riemann_lab_corpus @ {VAULT_RAG / 'chromadb'}\n")
114 fh.write(f"Cache LlamaIndex : {VAULT_RAG / 'llamaindex_cache'}\n")
115
112 - print(f"Nodes (chunks) indexés : {len(index.docstore.docs)}")
116 + print(f"Nodes (chunks) indexés : {n_chunks}")
117 print(f"Log écrit : {log_path}")
118
119
• Ingestion complète terminée et validée.
- 106 documents sources - 838 chunks dans la collection ChromaDB riemann_lab_corpus (/mnt/vault_rag/chromadb)
- 59 pages wiki courantes (hors Handoff.md)
- 44 fichiers de code (src/calculs/optimisation/**/*.{py,c,h}, hors .bak)
- 3 prompts archivés (bandeau ARCHIVE détecté)
- Cache LlamaIndex persiste sur /mnt/vault_rag/llamaindex_cache
- Log dans /mnt/vault_rag/agent_logs/ingestion_20260705_140201.log
- Script réutilisable : scripts/rag_ingest_corpus.py (non commité - dis-moi si tu veux que je le committe)
- Requête de vérification sur SEUIL_1NEWTON déjà concluante (retrouve le bon extrait de analyse_problemes_v13_v15.md)
Étape 3 du plan terminée. Prochaine étape ($5, étape 4) : le premier test de requête formel (« quel est le seuil SEUIL_1NEWTON et pourquoi ? ») - déjà effectivement validé ci-dessus, mais je peux le refaire proprement avec Ollama/mathstral en boucle RAG complète si tu veux enchaîner.
* Crunched for 3m 32s
> Committe le script rag_ingest_corpus.py
Committed 61ca640, ran 1 shell command
• Committé (61ca640) sur Riemann_Lab_C, non poussé - dis-moi si tu veux que je pousse aussi.
* Worked for 13s
> Pousse Riemann_Lab_C
Pushed to Riemann_Lab_C, ran 1 shell command
• Poussé (fe9fb7a..61ca640). Le script d'ingestion RAG est maintenant sur le remote.
* Crunched for 13s
> Mais le test de requête Ollama/mathstral sur SEUIL_1NEWTON
Sh 82% 1h53m 7d 25% 2d23h | Sonnet 5 (190.0k/1.0M) | cache 3m06s
Riemann_Lab_C Riemann_Lab_C
accept edits on (shift+tab to cycle) - for agents
L 10, col 53 Espaces : 4 UTF-8 LF Python defaultInterpreterPath: 3.12.3 Final 0 BLACKBOXAI Open Chat
```

La collection `riemann_lab_corpus` est remplie (59 pages wiki, 44 fichiers de code, 3 prompts ARCHIVE). La requête de vérification `SEUIL_1NEWTON` retrouve le bon extrait.

A.4 — Les fichiers créés sur le SSD (l'anatomie du vault)

```
riemann@zeta-lab: /mnt/vault_rag/llamaindex_cache
riemann@zeta-lab:/mnt/vault_rag$ ls
agent_logs chromadb corpus llamaindex_cache lost+found
riemann@zeta-lab:/mnt/vault_rag$ cd agent_logs/
riemann@zeta-lab:/mnt/vault_rag/agent_logs$ ls
ingestion_20260705_140201.log
riemann@zeta-lab:/mnt/vault_rag/agent_logs$ cd ../chromadb/
riemann@zeta-lab:/mnt/vault_rag/chromadb$ ls
1b199dce-ed16-4013-afe1-97cf6f6e6bed 940dc464-ecfe-49ae-bb5b-f947a38384a8 chroma.sqlite3
riemann@zeta-lab:/mnt/vault_rag/chromadb$ cd 1b199dce-ed16-4013-afe1-97cf6f6e6bed/ls
bash: cd: 1b199dce-ed16-4013-afe1-97cf6f6e6bed/ls: Aucun fichier ou dossier de ce nom
riemann@zeta-lab:/mnt/vault_rag/chromadb$ cd 940dc464-ecfe-49ae-bb5b-f947a38384a8/
riemann@zeta-lab:/mnt/vault_rag/chromadb/940dc464-ecfe-49ae-bb5b-f947a38384a8$ ls
data_level0.bin header.bin length.bin link_lists.bin
riemann@zeta-lab:/mnt/vault_rag/chromadb/940dc464-ecfe-49ae-bb5b-f947a38384a8$ cd ../1b199dce-ed16-4013-afe1-97cf6f6e6bed/
riemann@zeta-lab:/mnt/vault_rag/chromadb/1b199dce-ed16-4013-afe1-97cf6f6e6bed$ ls
data_level0.bin header.bin length.bin link_lists.bin
riemann@zeta-lab:/mnt/vault_rag/chromadb/1b199dce-ed16-4013-afe1-97cf6f6e6bed$ cd ../../corpus/
riemann@zeta-lab:/mnt/vault_rag/corpus$ ls
riemann@zeta-lab:/mnt/vault_rag/corpus$ cd ../llamaindex_cache/
riemann@zeta-lab:/mnt/vault_rag/llamaindex_cache$ ls
docstore.json graph_store.json image_vector_store.json index_store.json
riemann@zeta-lab:/mnt/vault_rag/llamaindex_cache$ cd ../lost+found/
bash: cd: ../lost+found/: Permission non accordée
riemann@zeta-lab:/mnt/vault_rag/llamaindex_cache$ ls
docstore.json graph_store.json image_vector_store.json index_store.json
riemann@zeta-lab:/mnt/vault_rag/llamaindex_cache$
```

Exploration du vault après ingestion : `chroma.sqlite3` (métastore), dossiers UUID avec `data_level0.bin` / `link_lists.bin` (vecteurs + HNSW), et les JSON du cache LlamaIndex.